



CAPÍTULO 10

Arquivos

Como um programa C grava e recupera informação em disco — em modo texto, legível, ou em modo binário, byte a byte — e como escolher a forma certa de organizar cada tipo de dado.



01 4E 6F

O que vamos ver

01 Abrir e fechar arquivos

fopen, os modos r/w/a/+, e por que sempre testar o retorno

02 Arquivos em modo texto

fscanf, fgetc, fgets para ler; fprintf, fputc para escrever

03 Estruturação de dados

caractere a caractere, linha a linha, palavras-chave — e sscanf

04 Arquivos em modo binário

fwrite, fread, e acesso direto a um registro com fseek

fopen devolve NULL quando falha

Todo código deve testar o retorno antes de usar o arquivo.

"r"

leitura — o arquivo precisa já existir

"w"

escrita — cria vazio, destrói o que já existia

"a"

escrita no final — preserva o conteúdo anterior

"+"

combinado a r/w — leitura e escrita no mesmo arquivo

01_COUNT_CHARS/MAIN.C

```
FILE *file = fopen("main.c", "r");

if (file == NULL)
{
    printf("Error opening file\n");
    return 1;
}
```

Três jeitos de ler, cada um com seu grão

Mesma família de funções do `stdio.h`, lendo de um `FILE*`.

`fgetc(fp)`

um caractere por chamada — o grão mais fino

`fgets(s, n, fp)`

uma linha inteira, até `'\n'` ou `n-1` caracteres

`fscanf(fp, fmt, ...)`

valores formatados, como um `scanf` que lê de arquivo

```
03_COUNT_LINES/MAIN.C
while (fgets(line, 200, file) != NULL)
{
    line_count++;
}
```

fprintf e fputc espelham a leitura

Dois arquivos de entrada, um de saída — três FILE* abertos ao mesmo tempo.



```
04_SUM_VECTORS/MAIN.C
fprintf(output_file, "%d\n", n1);
for (int i = 0; i < n1; i++)
{
    fprintf(output_file, "%d ", sum_vector[i]);
}
```

Como organizar o que está dentro do arquivo

A escolha depende da aplicação — os três exemplos deste módulo isolam cada mecânica

Caractere a caractere

fgetc num laço até EOF — ideal para contar ou classificar caracteres individuais.

```
01_count_chars
```

Linha a linha

fgets até NULL — cada linha vira uma unidade, útil combinada com sscanf.

```
03_count_lines
```

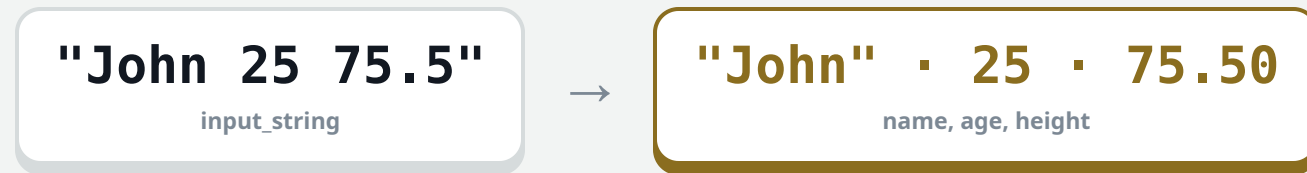
Palavras-chave

fscanf com %s captura um token por vez — a base para despachar por tipo de registro.

```
02_count_words
```

sscanf: fscanf para uma string em memória

O padrão real: fgets pega a linha inteira, sscanf extrai os campos dela.



```
05_STRING_PARSER/MAIN.C
char input_string[] = "John 25 75.5";
char name[50];
int age;
float height;

sscanf(input_string, "%s %d %f", name, &age, &height);
```

Os mesmos bytes da memória, sem tradução

`fwrite/fread` copiam blocos inteiros — sem nenhuma formatação de texto.

1 2 . 5 0

modo texto — "12.50",
legível, 5 bytes

VS

41 48 00 00

modo binário — os 4
bytes brutos do float
12.50

```
06_BINARY_RECORDS/MAIN.C
void salvar_produtos(
    const char *arquivo, Produto *produtos, int n)
{
    FILE *fp = fopen(arquivo, "wb");
    fwrite(produtos, sizeof(Produto), n, fp);
}
```

fseek: acesso direto a um registro

Como cada registro ocupa sempre o mesmo número de bytes, dá para pular direto até ele.



fseek(fp, 1 × sizeof(Produto), SEEK_SET) → salta direto para Produto[1], sem ler Produto[0]

```
06_BINARY_RECORDS/MAIN.C
Produto carregar_produto_por_indice(
    const char *arquivo, int indice)
{
    Produto p;
    FILE *fp = fopen(arquivo, "rb");
    fseek(fp, indice * sizeof(Produto), SEEK_SET);
    fread(&p, sizeof(Produto), 1, fp);
    fclose(fp);
    return p;
}
```

AGORA É COM VOCÊ

Explore os exemplos, no seu ritmo

Código-fonte, explicação linha a linha e simulação da execução passo a passo — sem precisar compilar nada. Depois, teste o que aprendeu no quiz.

 **Notas de Aula e Exemplos** thiagopx.github.io/ed/08_files